Rapport projet IA-Web-Big Data

Baptiste Richard – Gauthier Civilise - Océane Hasbi

II. Intelligence artificielle

Rapport projet IA-Web-Big Data 1
II. Intelligence artificielle 1
Besoins client 1 1
Analyse des besoins : 1
Choix du modèle : 2
Analyse des scores : 2
Péripéties : 3
Conclusion : 3
Besoin client 2 3
1 er partie : régression linéaire avec une variable 4
2ème partie : Random Forest avec une variable 4
3ème partie : Régression linéaire multi-features 5
4ème partie : Random Forest multi-features 5
5ème partie : Random Forest avec GridSearchCV 5
Script Python “age_arbre_py” 6
Besoin client 3 7
1. Analyse des données et préparation 8
2. Prétraitement des données 8
3. Modélisation et choix de l’algorithme 9
4. Évaluation et validation 11
5. Industrialisation et réutilisation 11
6. Technologies utilisées 12
Diagramme de Gantt 12

Besoins client 1

Analyse des besoins :

Le premier besoin du client 1 est de nettoyer la base de données. Cela a été fait lors de la partie I Big data.

Le client 1 soumet une demande d'apprentissage non supervisé. Un apprentissage non supervisé n'a pas d'étiquette permettant de vérifier la donnée analysée avec la réalité.

Nous avons déterminé que pour obtenir la répartition la plus fiable de la taille des arbres nous devons prendre uniquement la variable "haut_tot" qui définit la taille totale de l'arbre avec le tronc et les feuilles. Les autres paramètres semblent donc superflus pour répartir les arbres en fonction de leur taille.

Pour la visualisation de la carte ainsi que la catégorisation de la taille d'un arbre, un script permet de choisir le nombre de catégories souhaités.

Choix du modèle :

L'algorithme choisi pour regrouper les arbres par taille est le K-means. Cet algorithme fonctionne en regroupant des données proches ensemble.

Par exemple, sur un set de données contenant [12,13, 22, 23], la moyenne des données entre 12 et 13 est de 12,5 et celle entre 22 et 23 est de 22.5. Si on intègre un 18 dans ce dataset la moyenne dans le premier cluster (12 et 13) la moyenne sera de 14 tandis que si on l'intègre dans le cluster (22,23) la moyenne sera de 21. L'écart entre le cluster (12,13) et 18 est de 4 alors que l'écart entre le cluster (22,23) et 18 est de 3 donc 18 intègre le cluster (22,23). Et on compare la distance de chaque nouvelle valeur avec la moyenne des valeurs du cluster. Cela garantie une minimisation optimale de la variance au sein des clusters suivant le nombre de cluster.

C'est pour cette raison que la méthode des k-means a semblé être la meilleure pour séparer les données des tailles des arbres en 2, 3 ou plus, car elles se comparent entre elles et se classent par proximité.

Analyse des scores :

On remarque d'ailleurs dans notre cas, que la variance au sein d'un cluster est plus petite lorsque qu'il y a 3 clusters plutôt que 2. Cela peut s'expliquer cela par le fait que plus il y a de moyennes auxquelles une valeur peut se rapprocher, plus elle est précise. On peut aussi prendre en compte qu'il y a 5 arbres qui mesurent 37 mètres tandis qu'il y a 1765 arbres qui mesurent 7 mètres. Cela explique la variance supérieure dans la catégorie "arbres grands" par rapport à la catégorie "arbres petits", la moyenne de la taille des arbres sera influencée par le grand nombre d'arbres qui mesurent 7 mètres.

Cela est expliqué par différents scores : le silhouette Coefficient, le Calinski-Harabasz

Index et le Davies-Bouldin Index.

Pour le meilleur résultat de l'analyse silhouette on obtient un score de 0.65. Ce score est compris entre –1 et 1 et doit être le plus proche de 1 car il se sert d'indicateur de proximité des clusters. Plus le score se rapproche de 1, plus les données sont proches de leurs voisins et donc dans le cluster qui leur convient.

Pour le score de Calinski-Harabasz on cherche à obtenir un score le plus grand possible car il s'agit d'un rapport entre la variance intergroupe sur la variance intragroupe. Dans notre cas, nous obtenons un score de 36775.50 pour 5 clusters ce qui est logique car la variance intragroupe est plus petite tandis que la variance intergroupe augmente.

Le dernier indicateur analysé est le score de Davies-Bouldin. Il calcul les points moyens par cluster et détermine la moyenne des points et la compare au centre du cluster. Plus ce score se rapproche de 0 plus les valeurs dans le cluster sont proches et donc la séparation entre les clusters fiable. Dans notre cas, nous avons 0.51 de score ce qui se rapproche beaucoup de 0.

Péripéties :

Pour la génération de la carte, nous nous sommes rendu compte que les coordonnées ne sont pas référencées sur la norme GPS. Nous avons dû convertir depuis système de référence de coordonnées (CRS) 3949 au CRS universel pour la norme GPS avec les latitude et longitude 4326.

Une carte permettant de voir les arbres en fonction de leur taille suivant le nombre de catégories voulues est disponible suite à l'exécution du script.

Lors du clustering, le modèle ne sait pas si l'arbre est petit ou grand donc nous avons réalisé un mapping. Lors d'une analyse, nous nous sommes rendu compte que le cluster 0 correspondait aux arbres grands pour 2 clusters et cluster 1 aux arbres petits. Nous avons donc créé une colonne ou lorsqu'il y a écrit 0, la colonne "taille cluster 2" ai la valeur "petit"

Conclusion :

Ce modèle permet d'avoir une bonne répartition des données en fonction de leur taille malgré une variance plus élevée dans le cluster à l'extrémité supérieure. L'export d'un nouveau fichier CSV avec les clusters prédéfini permet de ne pas exécuter le notebook et de ne pas réentraîner le modèle à chaque lancement du script. Cependant, si le dataset change il faudra relancer au moins une fois le notebook contenant le modèle.

Besoin client 2

Le besoin client 2 consiste à développer un modèle d'intelligence artificielle capable de prédire l'âge estimé d'un arbre à partir de ses caractéristiques physiques et environnementales.

La variable cible est « age_estim », une valeur numérique, il s'agit donc d'un problème de régression supervisée et non de classification. Quatre étapes structurent ce travail : le choix et l'entraînement des algorithmes avec une variable (feature), et l'entraînement des algorithmes avec plusieurs variables (feature), puis l'utilisation GridSearchCV sur un modèle pour tester toutes les combinaisons possibles avec plusieurs hyperparamètres, et la préparation d'un script de prédiction réutilisable.

1 er partie : régression linéaire avec une variable

Le premier modèle supervisé utilisé est celui de la régression linéaire pour plusieurs raisons. Premièrement, c'est un modèle que l'on connait (déjà travailler en cours), ce modèle est simple et facile à interpréter. Enfin, nous avons fait une régression linéaire sur la partie statistique en R, cela nous permet de faire une première comparaison pour savoir si notre modèle est correct.

Le premier modèle de régression linéaire a été fait avec une "feature" d'entrée pour prédire l'âge estimé, tout comme le modèle fait en R, on utilise comme données d'entrée le diamètre du tronc de l'arbre.

Un nettoyage des données est refait sur ces deux variables, on garde seulement les arbres en "PLACE" car ils peuvent avoir des troncs de diamètre 0, car l'arbre est abattu, et on enlever les valeurs vides, ils nous restent 9393 lignes.

J'obtiens ainsi un RMSE de 12,85 ans et un R^2 de 0.583, ainsi ce modèle n'explique que la moitié des valeurs prédites.

2ème partie : Random Forest avec une variable

Le deuxième modèle utilisé est le Random Forest, toujours avec le diamètre du tronc comme unique variable d'entrée. Ce modèle est un algorithme d'ensemble qui combine plusieurs arbres de décision afin de produire une prédiction plus robuste et plus précise. Ce choix est stratégique : en testant d'abord le Random Forest dans sa configuration la plus simple, avec la même variable que la régression linéaire, cela permet d'établir une comparaison directe entre les deux algorithmes à données égales. De plus, le Random Forest étant un modèle qui repose sur des hyperparamètres,

l'utiliser dès cette étape prépare le terrain pour l'optimisation via GridSearchCV réalisée en partie 5. Il est généralement plus performant que la régression linéaire car il est capable de capturer des relations non-linéaires entre les variables.

Avec les mêmes données nettoyées (9393 arbres), on obtient un RMSE de 12.78 ans et un R^2 de 0.58. On constate une très légère amélioration par rapport à la régression linéaire simple, ce qui confirme que la relation entre le diamètre du tronc et l'âge estimé n'est pas entièrement linéaire. Cependant, une seule variable d'entrée reste insuffisante pour expliquer pleinement la variabilité de l'âge des arbres sur ce modèle.

3ème partie : Régression linéaire multi-features

Pour améliorer les performances du modèle, on enrichit les données d'entrée en ajoutant plusieurs variables numériques et catégorielles. Les variables numériques retenues sont le diamètre du tronc, la hauteur totale, la hauteur du tronc et le nombre de diagnostics. Les variables catégorielles sont le stade de développement, la situation, le port, le pied, le caractère remarquable et le feuillage.

Un pipeline de prétraitement est mis en place : les valeurs manquantes des variables numériques sont remplacées par la médiane, celles des variables catégorielles par la valeur la plus fréquente, et un encodage One-Hot est appliqué sur ces dernières pour les rendre utilisables par le modèle.

On obtient un RMSE de 9.65 ans et un R^2 de 0,765. L'ajout de ces nouvelles variables améliore significativement les résultats par rapport à la régression linéaire simple, ce qui montre que l'âge d'un arbre dépend de plusieurs facteurs et ne peut pas être réduit au seul diamètre du tronc.

4ème partie : Random Forest multi-features

On applique maintenant le Random Forest avec l'ensemble des mêmes variables d'entrée que la régression linéaire multi-features. Le pipeline de prétraitement est identique, ce qui garantit une comparaison équitable entre les deux modèles.

On obtient un RMSE de 7,58 ans et un R^2 de 0.855, ce qui constitue les meilleures performances observées jusqu'ici. Le Random Forest tire pleinement profit de la richesse des features en combinant les différents facteurs de manière non-linéaire.

L'analyse des importances des variables montre que “stadedev” est la plus déterminante dans la prédiction de l'âge, suivie de “trom_diam” et “haut_tonc.

5ème partie : Random Forest avec GridSearchCV

Le dernier modèle reprend le Random Forest multi-features en y ajoutant une optimisation des hyperparamètres via GridSearchCV. Cette méthode teste toutes les combinaisons possibles d'hyperparamètres grâce à une validation croisée à 5 folds, ce qui permet de trouver la configuration la plus performante sans risque de surapprentissage.

Les hyperparamètres explorés sont le nombre d'arbres (100, 200, 300), la profondeur maximale des arbres (None, 10, 20), le nombre minimal d'échantillons pour diviser un nœud (2, 5, 10), le nombre minimal d'échantillons dans une feuille (1, 2, 4) et le nombre de features considérées à chaque split (sqrt, log2). Au total, 810 combinaisons ont été testées.

Les meilleurs hyperparamètres trouvés sont :

modele__max_depth : 20
modele__max_features : log2
modele__min_samples_leaf : 1
modele__min_samples_split : 2
modele__n_estimators : 200

Le modèle optimisé atteint un RMSE de 7,5 ans et un R^2 de 0.8579, soit une amélioration de 0,029 par rapport au Random Forest de base. Ce résultat confirme que l'optimisation des hyperparamètres permet d'affiner les performances du modèle, même si le gain reste modeste lorsque le modèle de base est déjà bien configuré.

Script Python “age_arbre_py”

Faire la commande :
python age_arbre.py train --data Données_V4.csv

Génère model_rf_optimise.pkl dans le même dossier, puis :

***python age_arbre.py predict ***

***--tronc_diam 45 --haut_tot 12 --haut_tronc 2.5 --clc_nbr_diag 3 ***

***--fk_stadedev Adulte --fk_situation Alignement ***

***--fk_port Libre --fk_pied Asphalte ***

--remarquable Non --feuillage Persistant

Besoin client 3

Pour ce projet de système d'alerte pour les tempêtes, l'objectif principal était d'entraîner une IA capable de prédire l'état d'un arbre à partir de ses caractéristiques, afin d'identifier les arbres susceptibles de tomber lors d'une tempête.

1. Analyse des données et préparation

Les données contiennent plus de 30 variables décrivant les arbres : coordonnées géographiques, caractéristiques physiques, environnement d'implantation, informations administratives et historiques. La première étape a consisté à explorer les données pour comprendre leur structure, les nettoyer et identifier les variables utiles à la prédiction.

L'exploration a été réalisée avec pandas et numpy. Elle a montré que certaines colonnes n'étaient pas pertinentes pour la prédiction, comme les identifiants, les dates ou les variables redondantes. Elle a également révélé la présence de nombreuses variables catégorielles et un fort déséquilibre dans la variable cible fk_arb_etat.

Pour garder un modèle simple et justifiable, seules les variables jugées pertinentes ont été conservées :

coordonnées géographiques : X, Y

localisation : clc_quartier, clc_secteur

dimensions de l'arbre : haut_tot, haut_tronc, tronc_diam

caractéristiques descriptives : fk_stadedev, fk_port, fk_pied, fk_situation, fk_revetement

informations complémentaires : age_estim, villeca, feuillage, remarquable

La variable cible est l'état de l'arbre, fk_arb_etat.

L'analyse a aussi montré la présence de valeurs manquantes dans plusieurs colonnes, aussi bien numériques que catégorielles. Les lignes dont la cible fk_arb_etat était manquante ont été supprimées, car elles ne peuvent pas être utilisées dans un apprentissage supervisé. Pour les variables explicatives, la stratégie retenue a été :

- imputation par la médiane pour les variables numériques

- imputation par la modalité la plus fréquente pour les variables catégorielles

2. Prétraitement des données

Un point méthodologique important a été corrigé dans le notebook : le prétraitement ne doit pas être appris sur l'ensemble complet avant la séparation des données, sinon cela crée une fuite d'information entre le train et le test.

L'ordre retenu dans la version finale est le suivant :

- séparation des données en ensemble d'entraînement et ensemble de test
- apprentissage des objets de prétraitement uniquement sur l'ensemble d'entraînement
- transformation de X_train et X_test
- entraînement des modèles

Cette organisation évite la data leakage et rend l'évaluation plus fiable.

Les variables catégorielles ont été transformées avec OneHotEncoder de scikit-learn. Ce choix permet de convertir chaque modalité en variable binaire sans introduire d'ordre artificiel entre les catégories. L'encodeur est appris uniquement sur les données d'entraînement, puis réutilisé sur les données de test et sur les futures données de prédiction.

La question de la normalisation a également été étudiée. Finalement, aucune normalisation n'a été appliquée. Ce choix est cohérent avec les modèles retenus pour la comparaison, car ils sont tous les deux fondés sur des arbres de décision. Ces modèles ne sont pas sensibles à l'échelle des variables comme peuvent l'être la régression logistique, le SVM ou le KNN. Ajouter une normalisation aurait donc compliqué le pipeline sans apporter de gain clair.

Pour garantir la reproductibilité, les objets de prétraitement ont été sauvegardés avec joblib. Les fichiers sauvegardés correspondent aux objets réellement utilisés dans la version finale :

- l'encodeur OneHotEncoder
- l'imputeur catégoriel
- l'imputeur numérique
- la liste des colonnes attendues

Ces fichiers sont ensuite chargés dans le script de prédiction pour reproduire exactement le même prétraitement que lors de l'apprentissage.

3. Modélisation et choix de l'algorithme

Les données ont été séparées en un ensemble d'entraînement de 80 % et un ensemble de test de 20 %, avec une séparation stratifiée sur la variable cible. Cela permet de conserver une répartition comparable des classes dans les deux ensembles.

Suite à la recommandation du professeur, une comparaison entre deux modèles a été ajoutée afin de justifier le choix final :

- DecisionTreeClassifier
- RandomForestClassifier

Le DecisionTreeClassifier a été choisi comme modèle de comparaison car il est très simple à expliquer et repose sur la même logique de base qu'une forêt aléatoire. Le RandomForestClassifier a été retenu comme second modèle car il reste standard, robuste et adapté à ce type de données.

Une petite recherche d'hyperparamètres avec GridSearchCV a été réalisée pour chacun des deux modèles, avec des grilles simples et lisibles. L'objectif n'était pas de construire un système complexe, mais de comparer proprement deux approches standards.

Les résultats obtenus ont montré que le RandomForestClassifier est meilleur que le DecisionTreeClassifier sur le jeu de test. La forêt aléatoire obtient de meilleurs scores globaux, notamment en accuracy et en f1-score pondéré, et elle se montre plus robuste sur les classes minoritaires. Cela s'explique par le fait qu'une forêt aléatoire combine plusieurs arbres et limite davantage le surapprentissage qu'un arbre seul.

Le choix final du RandomForestClassifier est donc justifié par plusieurs arguments :

- c'est un algorithme standard de scikit-learn
- il est adapté à un mélange de variables numériques et catégorielles après encodage
- il fonctionne bien sans normalisation
- il donne de meilleurs résultats que l'arbre de décision seul

Optimisation des hyperparamètres :

L'optimisation a été réalisée avec GridSearchCV, en conservant une grille volontairement petite pour rester lisible et raisonnable :

```
param_grid = { 'n_estimators': [100, 200], 'max_depth': [10, 20, None], 'min_samples_split': [2, 5] }
```

Cette approche permet d'améliorer le modèle tout en gardant une logique simple. Les hyperparamètres testés sont directement interprétables :

- n_estimators : nombre d'arbres
- max_depth : profondeur maximale des arbres
- min_samples_split : nombre minimal d'observations pour diviser un nœud

Le besoin métier concerne le risque de chute ou d'intervention lors d'une tempête. Deux approches étaient envisageables conserver la classification multiclasses d'origine ou créer une version binaire opposant arbre stable et arbre à risque

Dans la version finale, la cible multiclasses fk_arb_etat a été conservée pour rester fidèle aux données disponibles et éviter d'introduire une règle de regroupement arbitraire. En revanche, une lecture métier complémentaire a été ajoutée dans le notebook en regroupant :

- EN PLACE : arbre stable

- autres classes : arbre potentiellement à risque ou nécessitant une intervention

Cette relecture permet d'interpréter les résultats selon l'objectif métier, tout en gardant un cadre méthodologique propre.

4. Évaluation et validation

L'évaluation du modèle repose sur les métriques suivantes :

- accuracy
- f1-score
- classification_report
- matrice de confusion

Le notebook insiste également sur un point important : la base est déséquilibrée, avec une classe EN PLACE très majoritaire. Dans ce contexte, l'accuracy seule peut être trompeuse. Il est donc nécessaire d'observer aussi le f1-score et surtout le recall des classes minoritaires, en particulier celles qui correspondent à des arbres plus sensibles du point de vue métier.

Les résultats obtenus sur l'ensemble de test montrent une bonne performance globale du modèle, avec une accuracy d'environ 0,95 et un F1-score pondéré d'environ 0,95

Le modèle reconnaît très bien la classe majoritaire EN PLACE, mais les performances sont plus modestes sur les classes rares comme ABATTU, ESSOUCHE ou REMPLACÉ. Ce constat est cohérent avec le déséquilibre des classes.

Pour rendre l'analyse plus proche du besoin métier, une seconde lecture des prédictions a été ajoutée dans le notebook sous la forme d'une opposition simple :

- stable
- risque

Cette relecture permet de mieux mettre en avant le rappel des arbres non stables, ce qui est particulièrement important dans une logique d'alerte. En pratique, dans ce type de projet, il est souvent préférable de détecter un maximum d'arbres à risque, quitte à accepter quelques faux positifs.

5. Industrialisation et réutilisation

Le script predict_tempete.py a été corrigé pour respecter une contrainte essentielle : il ne doit jamais relancer l'entraînement.

Dans sa version finale, le script :

- charge le modèle sauvegardé
- charge les objets de prétraitement sauvegardés
- remet les variables d'entrée dans le bon ordre
- applique les imputations et l'encodage

- renvoie directement la prédiction

Il propose deux modes d'utilisation avec un exemple pré-rempli et une saisie manuelle des caractéristiques via le terminal

6. Technologies utilisées

-Python 3.8+ : langage principal du projet

-pandas, numpy : manipulation et analyse des données

-scikit-learn : imputation, encodage, séparation train/test, modélisation, recherche d'hyperparamètres, évaluation

-joblib : sauvegarde du modèle et des objets de prétraitement

-matplotlib, seaborn : visualisation des matrices de confusion

-Jupyter Notebook : structuration pédagogique du travail et traçabilité des étapes

Conclusion

Ce projet a permis de mettre en place un système de prédiction simple, clair et réutilisable pour estimer l'état d'un arbre à partir de ses caractéristiques, dans une logique de prévention des risques liés aux tempêtes.
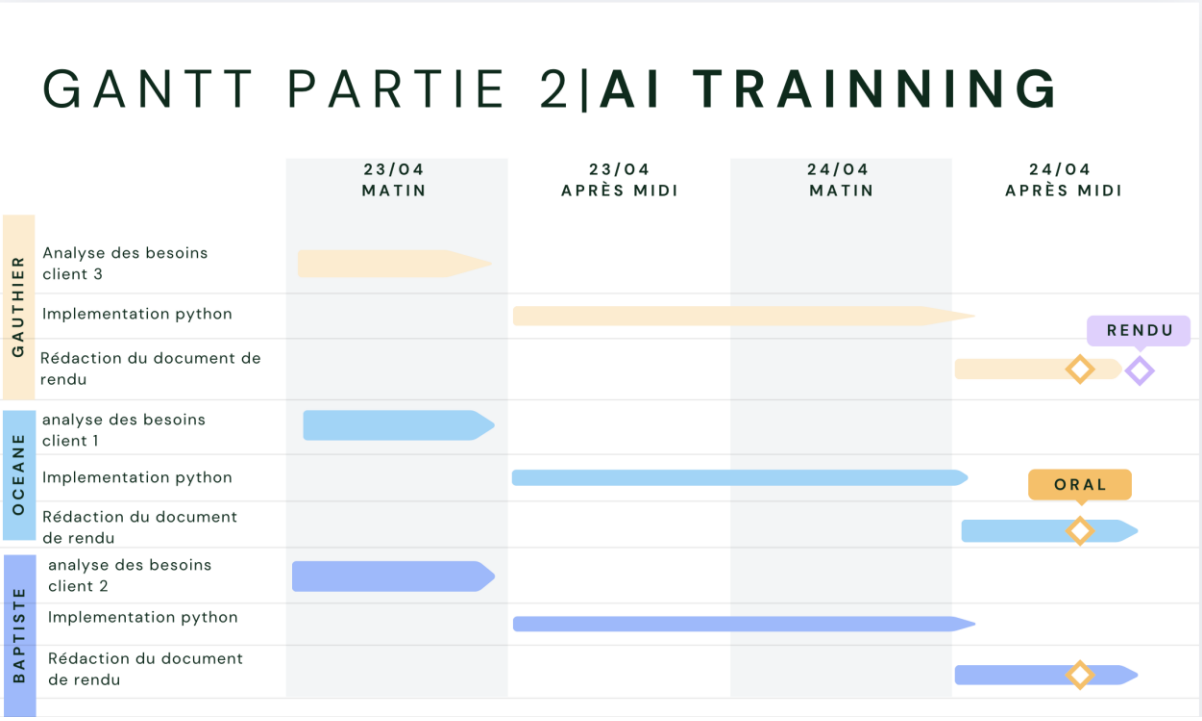
Diagramme de Gantt